

```

In [0]: # =====
# Project_Steam_part2 - Analyse des Genres
# Chargement et préparation des données
# =====

from pyspark.sql import functions as F

# Chargement depuis S3
df = spark.read.json("s3://full-stack-bigdata-datasets/Big_Data/Project_Steam/steam_game_output.json")

# Aplatissement
df_flat = df.select(
    F.col("id"),
    F.col("data.appid").alias("appid"),
    F.col("data.name").alias("name"),
    F.col("data.developer").alias("developer"),
    F.col("data.publisher").alias("publisher"),
    F.col("data.genre").alias("genre"),
    F.col("data.release_date").alias("release_date"),
    F.col("data.owners").alias("owners"),
    F.col("data.positive").alias("positive"),
    F.col("data.negative").alias("negative"),
    F.col("data.price").alias("price"),
    F.col("data.discount").alias("discount"),
    F.col("data.required_age").alias("required_age"),
    F.col("data.languages").alias("languages"),
    F.col("data.categories").alias("categories"),
    F.col("data.platforms.windows").alias("windows"),
    F.col("data.platforms.mac").alias("mac"),
    F.col("data.platforms.linux").alias("linux"),
    F.col("data.ccu").alias("ccu"),
    F.col("data.type").alias("type"),
)

# Nettoyage
df_games = df_flat.filter(F.col("type") == "game")
df_games = df_games.withColumn("price_eur", F.col("price") / 100)
df_games = df_games.withColumn("required_age", F.expr("try_cast(required_age as int)"))
df_games = df_games.withColumn(
    "positive_ratio",
    F.when(
        (F.col("positive") + F.col("negative")) > 0,
        F.round(F.col("positive") / (F.col("positive") + F.col("negative")) * 100, 2)
    ).otherwise(None)
)

print("Données prêtes ! Nombre de jeux :", df_games.count())

```

Données prêtes ! Nombre de jeux : 55690

```

In [0]: # =====
# ANALYSE DES GENRES
# Question 1 : Quels sont les genres les plus représentés ?
# =====

```

```
df_genres = df_games.filter(F.col("genre").isNotNull()) \
    .withColumn("genre_split", F.explode(F.split(F.col("genre"), ", "))) \
    .groupBy("genre_split") \
    .agg(F.count("appid").alias("nombre_jeux")) \
    .orderBy(F.desc("nombre_jeux"))

display(df_genres)
```

genre_split	nombre_jeux
Indie	39681
Action	23759
Casual	22086
Adventure	21431
Strategy	10895
Simulation	10836
RPG	9534
Early Access	6145
Free to Play	3393

Databricks visualization. Run in Databricks to view.

```
In [0]: # =====
# ANALYSE DES GENRES
# Question 2 : Quels genres ont le meilleur ratio d'avis positifs ?
# =====

df_genres_ratio = df_games.filter(F.col("genre").isNotNull()) \
    .withColumn("genre_split", F.explode(F.split(F.col("genre"), ", "))) \
    .filter((F.col("positive") + F.col("negative")) >= 100) \
    .groupBy("genre_split") \
    .agg(
        F.round(F.avg("positive_ratio"), 2).alias("ratio_moyen"),
        F.count("appid").alias("nombre_jeux")
    ) \
    .orderBy(F.desc("ratio_moyen"))

display(df_genres_ratio.select("genre_split", "ratio_moyen").limit(15))
```

genre_split	ratio_moyen
Game Development	82.58
Photo Editing	82.57
Web Publishing	81.91
Animation & Modeling	81.04
Design & Illustration	80.76
Sexual Content	79.76
Casual	79.71
Adventure	79.34
Indie	79.13

Databricks visualization. Run in Databricks to view.
Databricks visualization. Run in Databricks to view.

```
In [0]: #=====
# ANALYSE DES GENRES
# Question 3 : Quels sont les genres les plus lucratifs ?
# (on utilise les owners comme proxy des ventes)
# =====

# On extrait la valeur minimale des owners (ex: "200,000 .. 500,000" -> 200000)
df_genres_owners = df_games.filter(
    F.col("genre").isNotNull() & F.col("owners").isNotNull()) \
    .withColumn("genre_split", F.explode(F.split(F.col("genre"), ","))) \
    .withColumn("owners_min", F.regexp_replace(F.split(F.col("owners"), " \\. " ).getItem(0), ",", "").cast("long")) \
    .groupBy("genre_split") \
    .agg(F.sum("owners_min").alias("total_owners")) \
    .orderBy(F.desc("total_owners"))

display(df_genres_owners.limit(15))

<>:11: SyntaxWarning: invalid escape sequence '\.'
<>:11: SyntaxWarning: invalid escape sequence '\.'
/home/spark-97e3f544-cc78-4249-8615-43/.ipykernel/2103/command-7337978027093043-3016073354:11: SyntaxWarning: invalid escape sequence '\.'
.withColumn("owners_min",F.regexp_replace(F.split(F.col("owners"), " \\. " ).getItem(0), ",", "").cast("long")) \
<unknown>:11: SyntaxWarning: invalid escape sequence '\.'
```

genre_split	total_owners
Action	2865930000
Indie	1808090000
Adventure	1571480000
Free to Play	1210070000
RPG	1054110000
Strategy	965140000
Simulation	792030000
Casual	693320000
Massively Multiplayer	604860000

Databricks visualization. Run in Databricks to view.

```
In [0]: # =====
# ANALYSE DES PLATEFORMES
# Question 1 : La plupart des jeux sont-ils disponibles sur Windows/Mac/Linux ?
# =====

df_platforms = spark.createDataFrame([
    ("Windows", df_games.filter(F.col("windows") == True).count()),
    ("Mac", df_games.filter(F.col("mac") == True).count()),
    ("Linux", df_games.filter(F.col("linux") == True).count())
], ["plateforme", "nombre_jeux"])

display(df_platforms)
```

plateforme	nombre_jeux
Windows	55675
Mac	12769
Linux	8457

Databricks visualization. Run in Databricks to view.

```
In [0]: # =====
# ANALYSE DES PLATEFORMES
# Question 2 : Certains genres sont-ils disponibles de préférence sur certaines plateformes ?
# =====

df_genre_platform = df_games.filter(F.col("genre").isNotNull()) \
    .withColumn("genre_split", F.explode(F.split(F.col("genre"), ", "))) \
    .groupBy("genre_split") \
    .agg(
        F.round(F.avg(F.col("windows").cast("int")) * 100, 1).alias("pct_windows"),
        F.round(F.avg(F.col("mac").cast("int")) * 100, 1).alias("pct_mac"),
        F.round(F.avg(F.col("linux").cast("int")) * 100, 1).alias("pct_linux"),
```

```
F.count("appid").alias("nombre_jeux")
) \
.filter(F.col("nombre_jeux") >= 100) \
.orderBy(F.desc("pct_linux"))

display(df_genre_platform)
```

genre_split	pct_windows	pct_mac	pct_linux	nombre_jeux
	98.8	33.8	25.6	160
Game Development	100.0	32.7	22.0	159
Indie	100.0	25.0	17.6	39681
Strategy	100.0	27.6	16.8	10895
RPG	100.0	23.6	16.0	9534
Adventure	100.0	23.5	15.4	21431
Casual	100.0	23.2	15.0	22086
Action	100.0	19.2	14.2	23759
Simulation	100.0	22.5	14.1	10836

Databricks visualization. Run in Databricks to view.